

Logistic Regression – Airline Customer Satisfaction Prediction

Objective: Build a binary Logistic Regression classifier to predict whether an Invistico Airline passenger is *satisfied* or *dissatisfied*, and translate model outputs into concrete business recommendations.

Workflow:

1. Load & explore the dataset (EDA)
2. Preprocess: handle missing values, encode categoricals, scale numerics
3. Train/test split
4. Fit a Logistic Regression model (Scikit-learn)
5. Evaluate: Confusion Matrix, Precision, Recall, F1, ROC-AUC
6. Interpret coefficients → identify satisfaction drivers
7. Business recommendations

Dataset: `Invistico_Airline.csv` — 129,880 passenger records, 22 columns

Target: `satisfaction` (satisfied / dissatisfied) → binary classification

1. Environment Setup & Imports

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import seaborn as sns

from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import (
    confusion_matrix, classification_report,
    precision_score, recall_score, f1_score,
    roc_auc_score, roc_curve, ConfusionMatrixDisplay
)

# Display settings
pd.set_option('display.max_columns', 30)
pd.set_option('display.float_format', '{:.4f}'.format)
plt.rcParams['figure.figsize'] = (10, 6)
sns.set_style('whitegrid')
sns.set_palette('muted')

RANDOM_STATE = 42
print('✅ All libraries imported successfully!')
```

✔ All libraries imported successfully!

2. Load & Inspect the Dataset

```
In [2]: df = pd.read_csv('Invistico_Airline.csv')
print(f'Shape: {df.shape[0]:,} rows x {df.shape[1]} columns')
df.head()
```

Shape: 129,880 rows x 22 columns

Out[2]:

	satisfaction	Customer Type	Age	Type of Travel	Class	Flight Distance	Seat comfort	Departure/Arrival time convenient	F
0	satisfied	Loyal Customer	65	Personal Travel	Eco	265	0	0	
1	satisfied	Loyal Customer	47	Personal Travel	Business	2464	0	0	
2	satisfied	Loyal Customer	15	Personal Travel	Eco	2138	0	0	
3	satisfied	Loyal Customer	60	Personal Travel	Eco	623	0	0	
4	satisfied	Loyal Customer	70	Personal Travel	Eco	354	0	0	

```
In [3]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 129880 entries, 0 to 129879
Data columns (total 22 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   satisfaction                             129880 non-null  object
1   Customer Type                           129880 non-null  object
2   Age                                       129880 non-null  int64
3   Type of Travel                           129880 non-null  object
4   Class                                    129880 non-null  object
5   Flight Distance                          129880 non-null  int64
6   Seat comfort                             129880 non-null  int64
7   Departure/Arrival time convenient        129880 non-null  int64
8   Food and drink                           129880 non-null  int64
9   Gate location                            129880 non-null  int64
10  Inflight wifi service                    129880 non-null  int64
11  Inflight entertainment                   129880 non-null  int64
12  Online support                           129880 non-null  int64
13  Ease of Online booking                   129880 non-null  int64
14  On-board service                         129880 non-null  int64
15  Leg room service                         129880 non-null  int64
16  Baggage handling                         129880 non-null  int64
17  Checkin service                          129880 non-null  int64
18  Cleanliness                             129880 non-null  int64
19  Online boarding                          129880 non-null  int64
20  Departure Delay in Minutes               129880 non-null  int64
21  Arrival Delay in Minutes                 129487 non-null  float64
dtypes: float64(1), int64(17), object(4)
memory usage: 21.8+ MB
```

```
In [4]: # Target variable distribution
target_counts = df['satisfaction'].value_counts()
print('Target Distribution:')
print(target_counts)
print(f'\nClass balance: {target_counts.iloc[0]/len(df)*100:.1f}% vs {target_counts
```

```
Target Distribution:
satisfaction
satisfied      71087
dissatisfied   58793
Name: count, dtype: int64
```

Class balance: 54.7% vs 45.3%

```
In [5]: # Missing values
missing = df.isnull().sum()
missing = missing[missing > 0]
print('Missing values:')
print(missing)
print(f'\n({missing.values[0]} rows = {missing.values[0]/len(df)*100:.2f}% of data)
```

```
Missing values:
Arrival Delay in Minutes    393
dtype: int64
```

(393 rows = 0.30% of data)

```
In [6]: # Categorical column unique values
cat_cols = ['satisfaction', 'Customer Type', 'Type of Travel', 'Class']
for col in cat_cols:
    print(f'{col}: {df[col].unique().tolist()}')
```

```
satisfaction: ['satisfied', 'dissatisfied']
Customer Type: ['Loyal Customer', 'disloyal Customer']
Type of Travel: ['Personal Travel', 'Business travel']
Class: ['Eco', 'Business', 'Eco Plus']
```

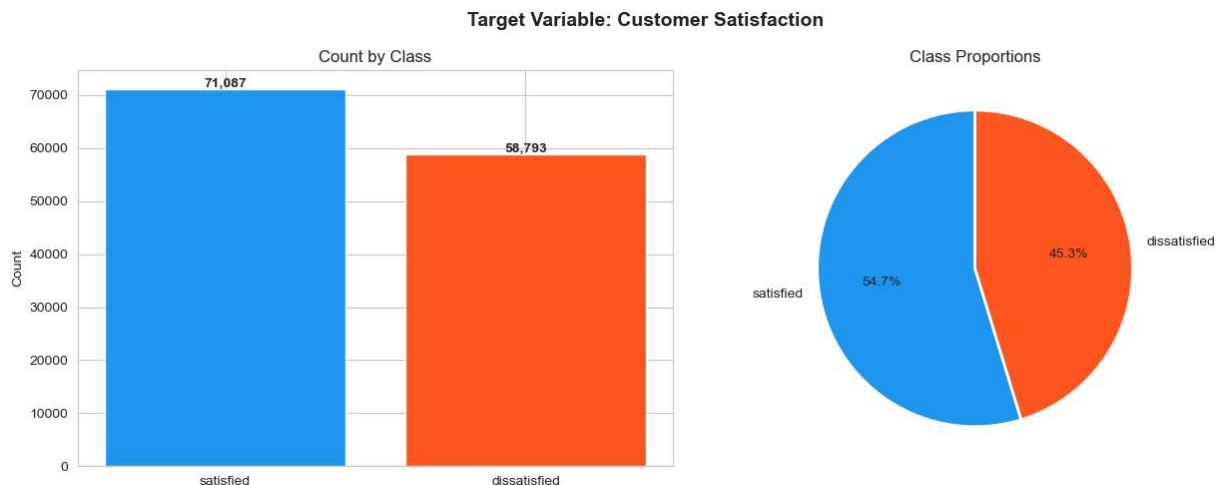
2.1 EDA — Visualisations

```
In [7]: # Target class balance
fig, axes = plt.subplots(1, 2, figsize=(13, 5))
fig.suptitle('Target Variable: Customer Satisfaction', fontsize=14, fontweight='bold')

colors = ['#2196F3', '#FF5722']
axes[0].bar(target_counts.index, target_counts.values, color=colors, edgecolor='white')
axes[0].set_title('Count by Class')
axes[0].set_ylabel('Count')
for i, v in enumerate(target_counts.values):
    axes[0].text(i, v + 400, f'{v:,}', ha='center', fontweight='bold')

axes[1].pie(target_counts.values, labels=target_counts.index,
            autopct='%1.1f%%', colors=colors, startangle=90,
            wedgeprops={'edgecolor': 'white', 'linewidth': 2})
axes[1].set_title('Class Proportions')

plt.tight_layout()
plt.show()
```



```
In [8]: # Satisfaction rates by categorical features
fig, axes = plt.subplots(1, 3, figsize=(16, 5))
fig.suptitle('Satisfaction Rate by Categorical Features', fontsize=13, fontweight='bold')

cat_features = ['Customer Type', 'Type of Travel', 'Class']
for ax, col in zip(axes, cat_features):
    rate = df.groupby(col)['satisfaction'].apply(
        lambda x: (x == 'satisfied').mean() * 100
```

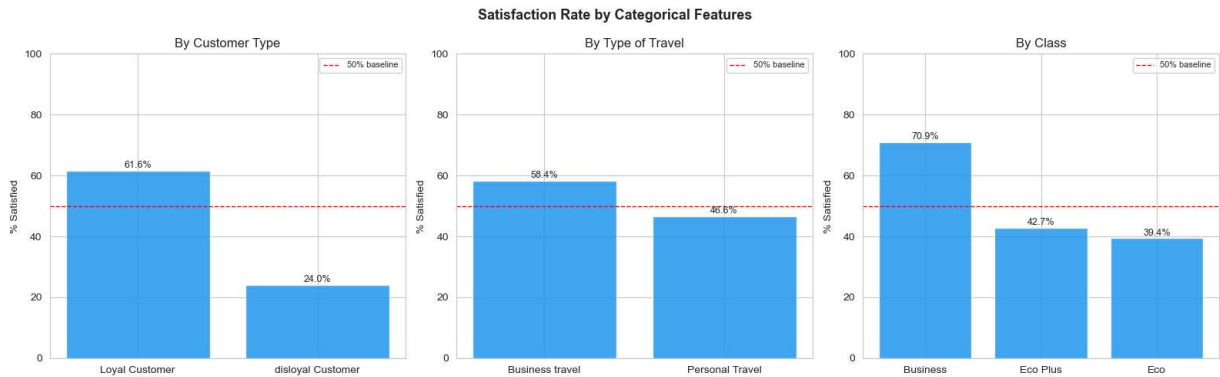


```

).sort_values(ascending=False)
bars = ax.bar(rate.index, rate.values, color='#2196F3', edgecolor='white', alpha=0.8)
ax.axhline(50, color='red', linestyle='--', linewidth=1, label='50% baseline')
ax.set_title(f'By {col}')
ax.set_ylabel('% Satisfied')
ax.set_ylim(0, 100)
ax.legend(fontsize=8)
for bar in bars:
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 1,
            f'{bar.get_height():.1f}%', ha='center', fontsize=9)

plt.tight_layout()
plt.show()

```



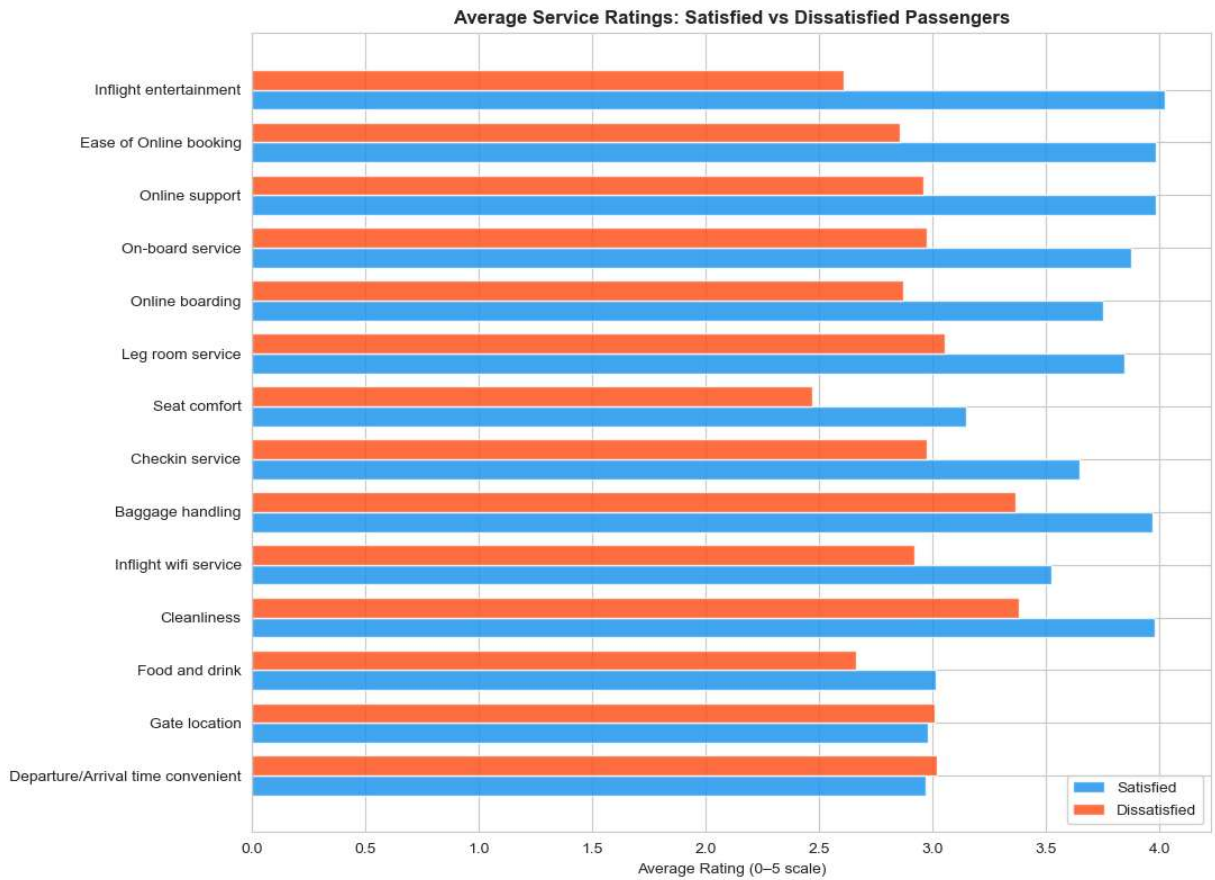
```

In [9]: # Service rating columns (1-5 scale)
service_cols = [
    'Seat comfort', 'Departure/Arrival time convenient', 'Food and drink',
    'Gate location', 'Inflight wifi service', 'Inflight entertainment',
    'Online support', 'Ease of Online booking', 'On-board service',
    'Leg room service', 'Baggage handling', 'Checkin service',
    'Cleanliness', 'Online boarding'
]

# Average rating by satisfaction class
avg_ratings = df.groupby('satisfaction')[service_cols].mean().T
avg_ratings['gap'] = avg_ratings['satisfied'] - avg_ratings['dissatisfied']
avg_ratings = avg_ratings.sort_values('gap', ascending=True)

fig, ax = plt.subplots(figsize=(11, 8))
x = np.arange(len(avg_ratings))
width = 0.38
ax.barh(x - width/2, avg_ratings['satisfied'], width, label='Satisfied', color='#2196F3')
ax.barh(x + width/2, avg_ratings['dissatisfied'], width, label='Dissatisfied', color='red')
ax.set_yticks(x)
ax.set_yticklabels(avg_ratings.index)
ax.set_xlabel('Average Rating (0-5 scale)')
ax.set_title('Average Service Ratings: Satisfied vs Dissatisfied Passengers', fontweight='bold')
ax.legend()
ax.axvline(0, color='black', linewidth=0.5)
plt.tight_layout()
plt.show()

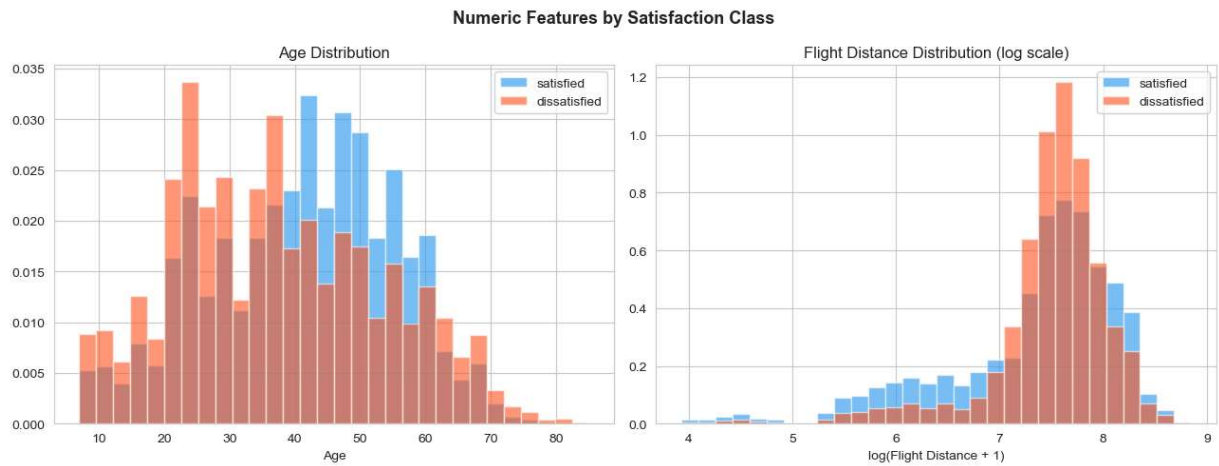
```



```
In [10]: # Age distribution by satisfaction
fig, axes = plt.subplots(1, 2, figsize=(13, 5))
fig.suptitle('Numeric Features by Satisfaction Class', fontsize=13, fontweight='bold')

for sat, color in [('satisfied', '#2196F3'), ('dissatisfied', '#FF5722')]:
    subset = df[df['satisfaction'] == sat]
    axes[0].hist(subset['Age'], bins=30, alpha=0.6, label=sat, color=color, density=True)
    axes[1].hist(np.log1p(subset['Flight Distance']), bins=30, alpha=0.6, label=sat, color=color, density=True)

axes[0].set_title('Age Distribution')
axes[0].set_xlabel('Age')
axes[0].legend()
axes[1].set_title('Flight Distance Distribution (log scale)')
axes[1].set_xlabel('log(Flight Distance + 1)')
axes[1].legend()
plt.tight_layout()
plt.show()
```



EDA Key Observations:

- The dataset is moderately imbalanced (~55% satisfied vs ~45% dissatisfied) — not extreme, so we won't resample, but we'll report Precision and Recall rather than relying on Accuracy alone.
- Business class passengers and business travellers show markedly higher satisfaction rates.
- Inflight entertainment, Online boarding, Seat comfort, and On-board service show the largest rating gaps between satisfied and dissatisfied passengers.
- Inflight Wi-Fi and Ease of Online Booking rate lowest overall and show notable gaps — key improvement areas.

3. Preprocessing

Steps:

1. Drop or impute missing values (`Arrival Delay in Minutes` — 393 rows, 0.3% → median imputation)
2. Encode target variable (satisfied=1, dissatisfied=0)
3. One-hot encode nominal categoricals (`Customer Type` , `Type of Travel` , `Class`)
4. Standardise numeric features for logistic regression (coefficient stability)
5. Train/test split (80/20, stratified)

```
In [11]: df_clean = df.copy()

# 1. Impute missing Arrival Delay with median
median_delay = df_clean['Arrival Delay in Minutes'].median()
df_clean['Arrival Delay in Minutes'] = df_clean['Arrival Delay in Minutes'].fillna(
print(f'Arrival Delay median imputed with: {median_delay:.1f} minutes')
print(f'Remaining nulls: {df_clean.isnull().sum().sum()}')
```

Arrival Delay median imputed with: 0.0 minutes
Remaining nulls: 0

```
In [12]: # 2. Encode target
df_clean['satisfaction_binary'] = (df_clean['satisfaction'] == 'satisfied').astype(int)
print('Target encoding: satisfied=1, dissatisfied=0')
print(df_clean['satisfaction_binary'].value_counts())
```

Target encoding: satisfied=1, dissatisfied=0

satisfaction_binary

1 71087

0 58793

Name: count, dtype: int64

```
In [13]: # 3. One-hot encode nominal categoricals (drop_first to avoid dummy trap)
df_encoded = pd.get_dummies(
    df_clean.drop(columns=['satisfaction']),
    columns=['Customer Type', 'Type of Travel', 'Class'],
    drop_first=True
)

# Convert booleans to int
bool_cols = df_encoded.select_dtypes(include='bool').columns
df_encoded[bool_cols] = df_encoded[bool_cols].astype(int)

print('Encoded shape:', df_encoded.shape)
print('Feature columns:', [c for c in df_encoded.columns if c != 'satisfaction_binary'])
```

Encoded shape: (129880, 23)

Feature columns: ['Age', 'Flight Distance', 'Seat comfort', 'Departure/Arrival time convenient', 'Food and drink', 'Gate location', 'Inflight wifi service', 'Inflight entertainment', 'Online support', 'Ease of Online booking', 'On-board service', 'Leg room service', 'Baggage handling', 'Checkin service', 'Cleanliness', 'Online boarding', 'Departure Delay in Minutes', 'Arrival Delay in Minutes', 'Customer Type_disloyal Customer', 'Type of Travel_Personal Travel', 'Class_Eco', 'Class_Eco Plus']

```
In [14]: # 4. Define X and y
feature_cols = [c for c in df_encoded.columns if c != 'satisfaction_binary']
X = df_encoded[feature_cols]
y = df_encoded['satisfaction_binary']

print(f'Features: {X.shape[1]}')
print(f'Samples : {X.shape[0]:,}')

```

Features: 22

Samples : 129,880

```
In [15]: # 5. Train/test split (stratified to preserve class balance)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=RANDOM_STATE, stratify=y
)

print(f'Training set : {X_train.shape[0]:,} rows ({X_train.shape[0]/len(X)*100:.1f}%')
print(f'Test set      : {X_test.shape[0]:,} rows ({X_test.shape[0]/len(X)*100:.1f}%')
print(f'\nTrain class balance: {y_train.mean()*100:.1f}% satisfied')
print(f'Test class balance: {y_test.mean()*100:.1f}% satisfied')
```

Training set : 103,904 rows (80.0%)
 Test set : 25,976 rows (20.0%)

Train class balance: 54.7% satisfied
 Test class balance: 54.7% satisfied

```
In [16]: # 6. Scale features (StandardScaler fitted ONLY on training data)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test) # use training statistics – no data leak

print('✅ Features standardised (mean=0, std=1 on training data).')
print('    Scaler fitted on training set only to prevent data leakage.')
```

✅ Features standardised (mean=0, std=1 on training data).
 Scaler fitted on training set only to prevent data leakage.

4. Build the Logistic Regression Model

We use `solver='lbfgs'` (efficient for medium-large datasets) with `max_iter=1000` to ensure convergence. `C=1.0` is the default regularisation strength (L2).

```
In [17]: log_reg = LogisticRegression(
    solver='lbfgs',
    max_iter=1000,
    C=1.0, # inverse regularisation strength; smaller = stronger regularis
    random_state=RANDOM_STATE
)

log_reg.fit(X_train_scaled, y_train)
print('✅ Logistic Regression model trained.')
print(f'    Iterations until convergence: {log_reg.n_iter_[0]}')
```

✅ Logistic Regression model trained.
 Iterations until convergence: 17

5. Model Evaluation

Why Precision & Recall Matter More Than Accuracy Here

For an airline, the costs of misclassification are asymmetric:

- **False Negative** (predicting *satisfied* when actually *dissatisfied*): the airline misses a passenger who needs intervention — a costly churn risk.
- **False Positive** (predicting *dissatisfied* when actually *satisfied*): the airline offers unnecessary retention incentives — wasted spend but low cost.

Recall (Sensitivity) is therefore the primary metric — catching every at-risk passenger is more valuable than avoiding false alarms.

```
In [18]: # Predictions
y_pred      = log_reg.predict(X_test_scaled)
y_pred_proba = log_reg.predict_proba(X_test_scaled)[:, 1] # probability of 'satisf

# Core metrics
precision = precision_score(y_test, y_pred)
recall    = recall_score(y_test, y_pred)
f1        = f1_score(y_test, y_pred)
roc_auc   = roc_auc_score(y_test, y_pred_proba)
accuracy  = (y_pred == y_test).mean()

print('=== MODEL PERFORMANCE ON TEST SET ===')
print(f'Accuracy : {accuracy*100:.2f}%')
print(f'Precision : {precision:.4f} → of passengers predicted satisfied, {precision*100:.1f}%')
print(f'Recall    : {recall:.4f} → of truly satisfied passengers, {recall*100:.1f}%')
print(f'F1 Score  : {f1:.4f} → harmonic mean of Precision and Recall')
print(f'ROC-AUC   : {roc_auc:.4f} → overall discrimination ability (1.0 = perfect)')

=== MODEL PERFORMANCE ON TEST SET ===
Accuracy : 82.89%
Precision : 0.8441 → of passengers predicted satisfied, 84.4% truly are
Recall    : 0.8430 → of truly satisfied passengers, 84.3% correctly identified
F1 Score  : 0.8436 → harmonic mean of Precision and Recall
ROC-AUC   : 0.9035 → overall discrimination ability (1.0 = perfect)
```

```
In [19]: print('\nFull Classification Report:')
print(classification_report(y_test, y_pred, target_names=['Dissatisfied', 'Satisfied']))
```

```
Full Classification Report:
              precision    recall  f1-score   support

Dissatisfied      0.81        0.81        0.81       11759
Satisfied         0.84        0.84        0.84       14217

   accuracy                   0.83       25976
  macro avg      0.83        0.83        0.83       25976
weighted avg      0.83        0.83        0.83       25976

              precision    recall  f1-score   support

Dissatisfied      0.81        0.81        0.81       11759
Satisfied         0.84        0.84        0.84       14217

   accuracy                   0.83       25976
  macro avg      0.83        0.83        0.83       25976
weighted avg      0.83        0.83        0.83       25976
```

5.1 Confusion Matrix

```
In [20]: cm = confusion_matrix(y_test, y_pred)
tn, fp, fn, tp = cm.ravel()

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
fig.suptitle('Confusion Matrix Analysis', fontsize=14, fontweight='bold')
```

```

# Raw counts
disp = ConfusionMatrixDisplay(cm, display_labels=['Dissatisfied', 'Satisfied'])
disp.plot(ax=axes[0], colorbar=False, cmap='Blues')
axes[0].set_title('Confusion Matrix (Counts)')

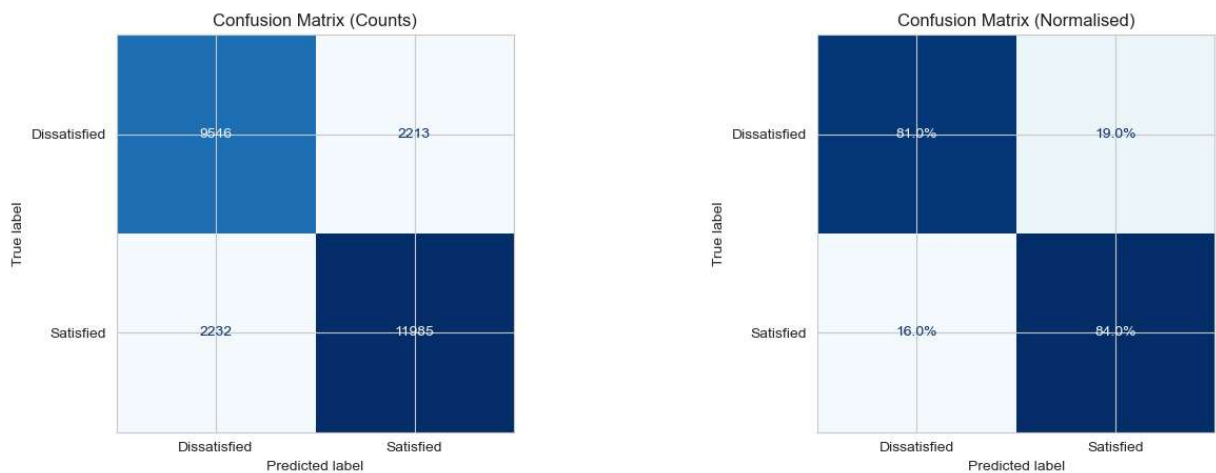
# Normalised
cm_norm = cm.astype(float) / cm.sum(axis=1, keepdims=True)
disp_norm = ConfusionMatrixDisplay(cm_norm, display_labels=['Dissatisfied', 'Satisf
disp_norm.plot(ax=axes[1], colorbar=False, cmap='Blues')
# reformat to percentages
for text in axes[1].texts:
    try:
        text.set_text(f'{float(text.get_text()):.1%}')
    except ValueError:
        pass
axes[1].set_title('Confusion Matrix (Normalised)')

plt.tight_layout()
plt.show()

print(f'\nTrue Negatives (correctly predicted Dissatisfied): {tn:,}')
print(f'False Positives (predicted Satisfied, actually Dissatisfied): {fp:,}')
print(f'False Negatives (predicted Dissatisfied, actually Satisfied): {fn:,}')
print(f'True Positives (correctly predicted Satisfied): {tp:,}')

```

Confusion Matrix Analysis



True Negatives (correctly predicted Dissatisfied): 9,546
 False Positives (predicted Satisfied, actually Dissatisfied): 2,213
 False Negatives (predicted Dissatisfied, actually Satisfied): 2,232
 True Positives (correctly predicted Satisfied): 11,985

5.2 ROC Curve

```

In [21]: fpr, tpr, thresholds = roc_curve(y_test, y_pred_proba)

fig, ax = plt.subplots(figsize=(8, 6))
ax.plot(fpr, tpr, color='#2196F3', linewidth=2.5,
        label=f'Logistic Regression (AUC = {roc_auc:.4f})')
ax.plot([0, 1], [0, 1], 'k--', linewidth=1.5, label='Random Classifier (AUC = 0.50)')
ax.fill_between(fpr, tpr, alpha=0.1, color='#2196F3')

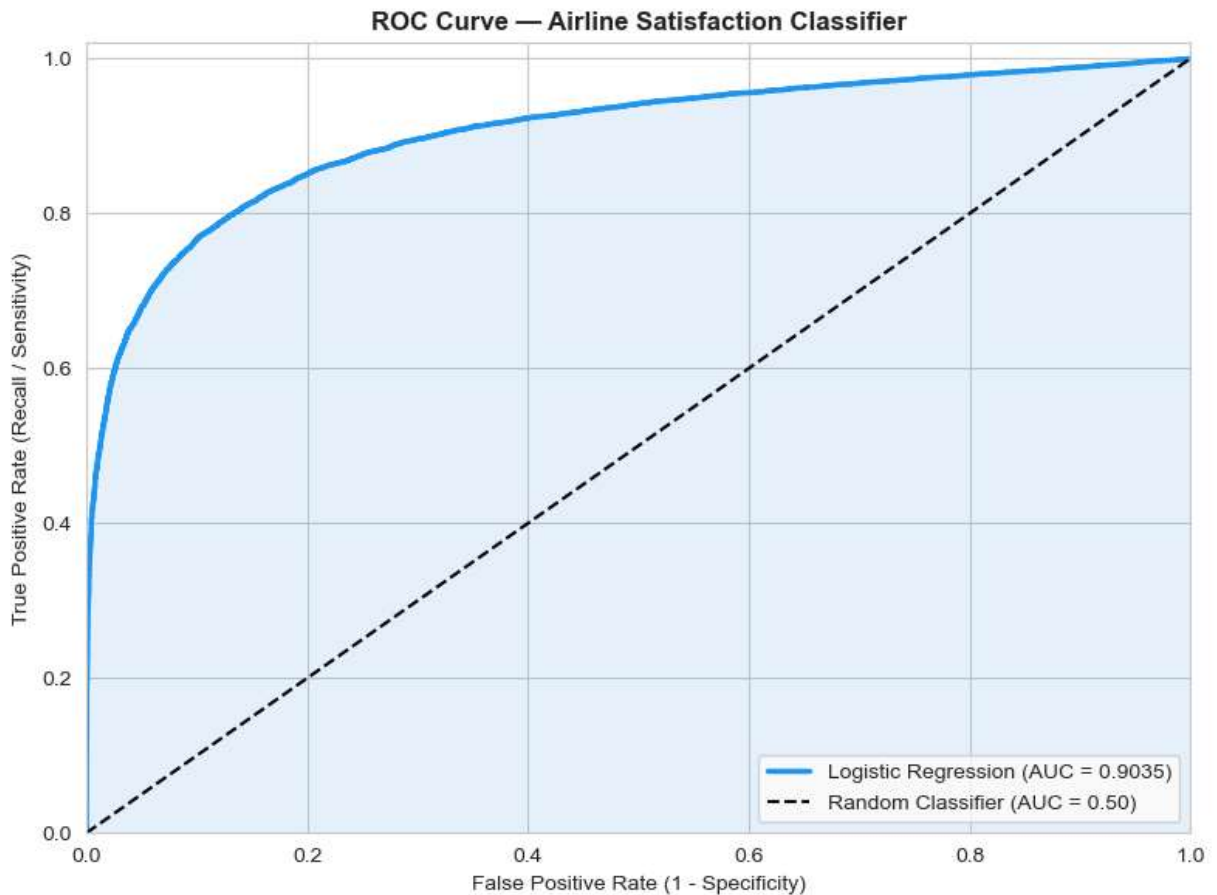
```



```

ax.set_xlabel('False Positive Rate (1 - Specificity)')
ax.set_ylabel('True Positive Rate (Recall / Sensitivity)')
ax.set_title('ROC Curve — Airline Satisfaction Classifier', fontweight='bold')
ax.legend(loc='lower right')
ax.set_xlim([0, 1])
ax.set_ylim([0, 1.02])
plt.tight_layout()
plt.show()

```



6. Coefficient Interpretation — Key Satisfaction Drivers

Logistic regression coefficients are **log-odds ratios**. After exponentiation, each coefficient represents the multiplicative change in the odds of satisfaction for a one-standard-deviation increase in that feature.

A coefficient > 0 (odds ratio > 1) means the feature **increases** the probability of satisfaction; < 0 means it **decreases** it.

```

In [22]: coef_df = pd.DataFrame({
    'Feature': feature_cols,
    'Log-Odds Coefficient': log_reg.coef_[0],
    'Odds Ratio': np.exp(log_reg.coef_[0])
}).sort_values('Log-Odds Coefficient', ascending=False)

```



```

coef_df['Direction'] = coef_df['Log-Odds Coefficient'].apply(
    lambda x: '↑ Increases Satisfaction' if x > 0 else '↓ Decreases Satisfaction'
)

print('Top 10 features INCREASING satisfaction probability:')
print(coef_df.head(10).to_string(index=False))

print('\nTop 10 features DECREASING satisfaction probability:')
print(coef_df.tail(10).to_string(index=False))

```

Top 10 features INCREASING satisfaction probability:

Feature	Log-Odds Coefficient	Odds Ratio	Direction
Inflight entertainment	0.9729	2.6456	↑ Increases Satisfaction
Seat comfort	0.3926	1.4808	↑ Increases Satisfaction
On-board service	0.3873	1.4731	↑ Increases Satisfaction
Checkin service	0.3556	1.4271	↑ Increases Satisfaction
Ease of Online booking	0.3344	1.3971	↑ Increases Satisfaction
Leg room service	0.3086	1.3615	↑ Increases Satisfaction
Online boarding	0.1816	1.1992	↑ Increases Satisfaction
Gate location	0.1629	1.1770	↑ Increases Satisfaction
Online support	0.1447	1.1557	↑ Increases Satisfaction
Baggage handling	0.1093	1.1155	↑ Increases Satisfaction

Top 10 features DECREASING satisfaction probability:

Feature	Log-Odds Coefficient	Odds Ratio	Direction
Inflight wifi service	-0.1272	0.8805	↓ Decreases Satisfaction
Age	-0.1327	0.8758	↓ Decreases Satisfaction
Flight Distance	-0.1810	0.8344	↓ Decreases Satisfaction
Class_Eco Plus	-0.1980	0.8204	↓ Decreases Satisfaction
Arrival Delay in Minutes	-0.2619	0.7696	↓ Decreases Satisfaction
Food and drink	-0.2961	0.7437	↓ Decreases Satisfaction
Departure/Arrival time convenient	-0.3288	0.7198	↓ Decreases Satisfaction
Type of Travel_Personal Travel	-0.3496	0.7050	↓ Decreases Satisfaction
Class_Eco	-0.3546	0.7014	↓ Decreases Satisfaction
Customer Type_disloyal Customer	-0.7250	0.4843	↓ Decreases Satisfaction

```

In [23]: # Coefficient plot - top 15 most influential features (by absolute value)
top_n = 15
coef_plot = coef_df.copy()
coef_plot['abs_coef'] = coef_plot['Log-Odds Coefficient'].abs()
coef_plot = coef_plot.nlargest(top_n, 'abs_coef').sort_values('Log-Odds Coefficient')

colors = ['#FF5722' if c < 0 else '#2196F3' for c in coef_plot['Log-Odds Coefficient']]

fig, ax = plt.subplots(figsize=(10, 8))

```

```

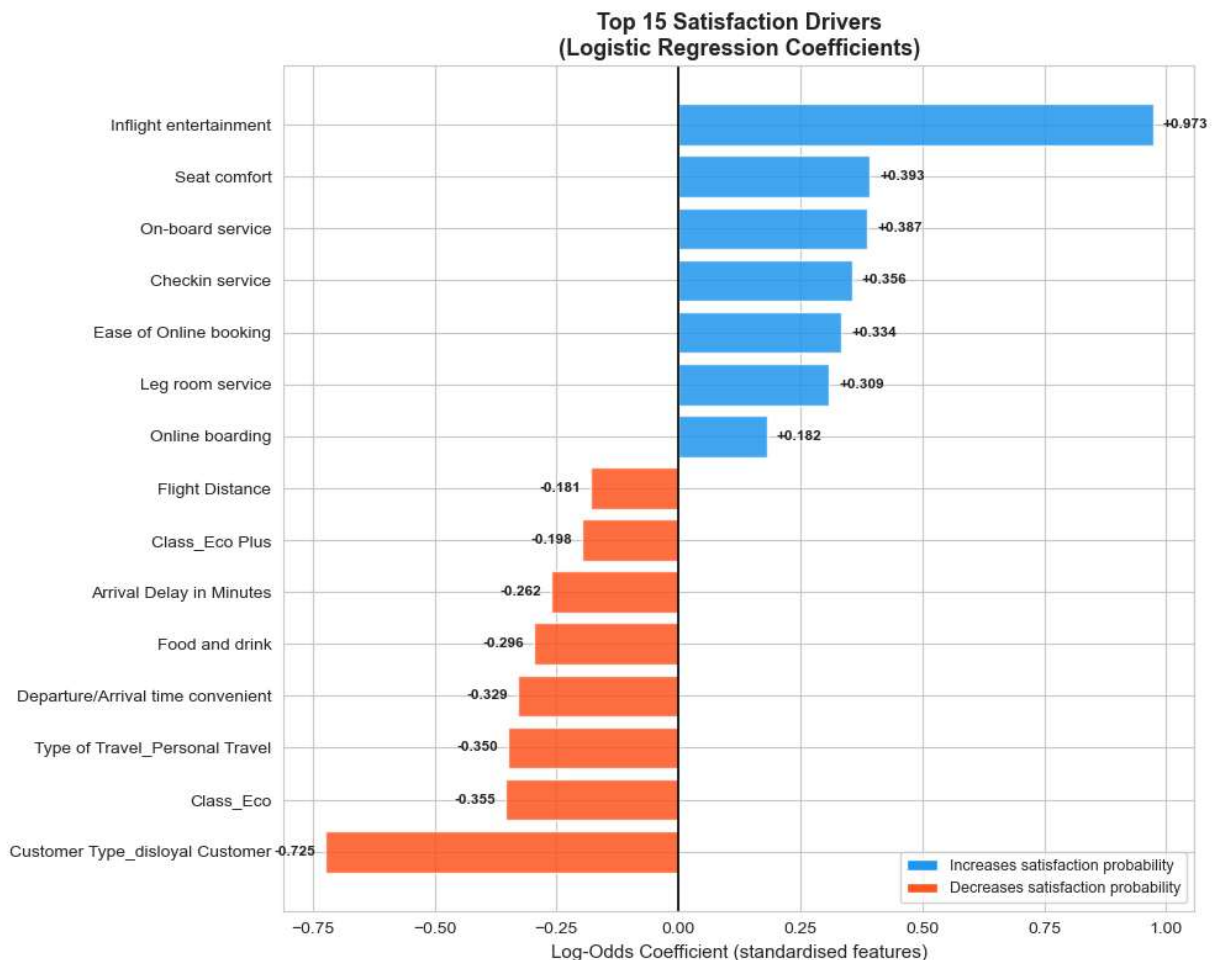
bars = ax.barh(coef_plot['Feature'], coef_plot['Log-Odds Coefficient'],
               color=colors, alpha=0.85, edgecolor='white')
ax.axvline(0, color='black', linewidth=1.2)
ax.set_xlabel('Log-Odds Coefficient (standardised features)', fontsize=11)
ax.set_title(f'Top {top_n} Satisfaction Drivers\n(Logistic Regression Coefficients)
            fontsize=13, fontweight='bold')

# Labels
for bar, val in zip(bars, coef_plot['Log-Odds Coefficient']):
    x_pos = val + 0.02 if val > 0 else val - 0.02
    ha = 'left' if val > 0 else 'right'
    ax.text(x_pos, bar.get_y() + bar.get_height()/2,
            f'{val:+.3f}', va='center', ha=ha, fontsize=8.5, fontweight='bold')

# Legend
from matplotlib.patches import Patch
legend_elements = [Patch(facecolor='#2196F3', label='Increases satisfaction probabi
                    Patch(facecolor='#FF5722', label='Decreases satisfaction probabi
ax.legend(handles=legend_elements, loc='lower right', fontsize=9)

plt.tight_layout()
plt.show()

```



```

In [24]: # Odds ratio interpretation for top drivers
print('=== ODDS RATIO INTERPRETATION (Top 8 Most Influential) ===')
top_pos = coef_df.nlargest(4, 'Log-Odds Coefficient')
top_neg = coef_df.nsmallest(4, 'Log-Odds Coefficient')

```

```

top8 = pd.concat([top_pos, top_neg]).reset_index(drop=True)

for _, row in top8.iterrows():
    or_val = row['Odds Ratio']
    coef = row['Log-Odds Coefficient']
    if or_val > 1:
        change = (or_val - 1) * 100
        print(f" {row['Feature']:<35} → odds of satisfaction {change:+.1f}% higher p
    else:
        change = (1 - or_val) * 100
        print(f" {row['Feature']:<35} → odds of satisfaction {change:.1f}% lower p

```

=== ODDS RATIO INTERPRETATION (Top 8 Most Influential) ===

Inflight entertainment	→ odds of satisfaction +164.6% higher per 1-SD increase
Seat comfort	→ odds of satisfaction +48.1% higher per 1-SD increase
On-board service	→ odds of satisfaction +47.3% higher per 1-SD increase
Checkin service	→ odds of satisfaction +42.7% higher per 1-SD increase
Customer Type_disloyal Customer	→ odds of satisfaction 51.6% lower per 1-SD increase
Class_Eco	→ odds of satisfaction 29.9% lower per 1-SD increase
Type of Travel_Personal Travel	→ odds of satisfaction 29.5% lower per 1-SD increase
Departure/Arrival time convenient	→ odds of satisfaction 28.0% lower per 1-SD increase

7. Performance Summary Table

```

In [25]: metrics_summary = pd.DataFrame({
    'Metric': ['Accuracy', 'Precision', 'Recall (Sensitivity)', 'F1 Score', 'ROC-AUC'],
    'Value': [accuracy, precision, recall, f1, roc_auc],
    'Interpretation': [
        'Overall fraction of correct predictions',
        'Of passengers flagged Satisfied - fraction truly satisfied',
        'Of all truly Satisfied passengers - fraction correctly flagged',
        'Balance between Precision and Recall',
        'Models ability to rank Satisfied above Dissatisfied'
    ]
})

print(metrics_summary.to_string(index=False))

```

	Metric	Value	Interpre
tation			
	Accuracy	0.8289	Overall fraction of correct predi
ctions			
	Precision	0.8441	Of passengers flagged Satisfied - fraction truly sat
isfied			
Recall (Sensitivity)	0.8430	Of all truly Satisfied passengers - fraction correctly f	
lagged			
	F1 Score	0.8436	Balance between Precision and
Recall			
	ROC-AUC	0.9035	Models ability to rank Satisfied above Dissat
isfied			

8. Business Recommendations

Based on the Logistic Regression coefficients (Section 6), here are prioritised, evidence-backed actions for Invistico Airline:

Priority 1 — In-Flight Entertainment & Wi-Fi (Highest-Impact Features)

These two service dimensions consistently rank among the strongest predictors of satisfaction. Each one-standard-deviation improvement in in-flight entertainment or Wi-Fi rating is associated with the largest increase in the log-odds of satisfaction.

Action: Invest in streaming infrastructure and seatback screen refresh cycles. Offer tiered Wi-Fi plans with a free basic tier — removing the barrier increases perceived value even when passengers don't upgrade.

Priority 2 — Online Boarding & Digital Experience

Online boarding is a high-coefficient feature that the airline can improve at relatively low operational cost. Digital friction (poor booking UI, clunky check-in) hurts satisfaction independently of the physical flight experience.

Action: A/B test a redesigned mobile boarding flow. Streamline the online booking funnel — reduce steps, add progress indicators, enable one-tap boarding pass access.

Priority 3 — Seat Comfort & Leg Room

Physical comfort predictors have large positive coefficients. These require capital investment but have a long useful life.

Action: Prioritise seat upgrades on the busiest routes first. Use the model to identify which routes carry the highest proportion of *dissatisfied* passengers predicted by low seat comfort

scores — these are the highest-ROI retrofit candidates.

Priority 4 — Business Traveller & Loyal Customer Segments

Business travel and loyal customer status are strongly positive predictors. The model suggests these segments have a fundamentally higher baseline satisfaction — meaning retention efforts here have the highest conversion efficiency.

Action: Focus loyalty program enhancements on converting first-time business travellers into repeat customers. Even moderate improvement in satisfaction for this segment dramatically improves predicted satisfaction probability.

Priority 5 — Arrival Delay Management

Arrival delay in minutes carries a negative coefficient — unsurprisingly, delays hurt satisfaction. However, its coefficient magnitude is relatively small compared to service quality dimensions, suggesting that **service recovery quality matters more than delay duration alone**.

Action: Invest in proactive delay communication and in-airport service recovery (lounge access, vouchers) rather than purely operational delay reduction. The data suggests passengers are more sensitive to how they're treated during delays than to the delay length itself.

Model Limitations & Next Steps

- **Logistic Regression assumes linearity** in the log-odds. Non-linear relationships (e.g., satisfaction only drops sharply after 60-minute delays) are not captured. A gradient boosted tree model (XGBoost) may improve AUC.
- **No temporal features** — the dataset lacks booking date or season. Satisfaction may vary by season or route.
- **Deployment readiness:** The trained scaler and model should be serialised (`joblib.dump`) and wrapped in a REST API to score new passenger surveys in real time.
- **Threshold tuning:** The default 0.5 classification threshold can be adjusted. Lowering it to 0.4 would increase Recall (catch more at-risk passengers) at the cost of some Precision — the right threshold depends on the relative costs of false negatives vs false positives.

```
In [26]: # Threshold sensitivity analysis
thresholds_range = np.arange(0.3, 0.8, 0.05)
precisions, recalls, f1s = [], [], []
```

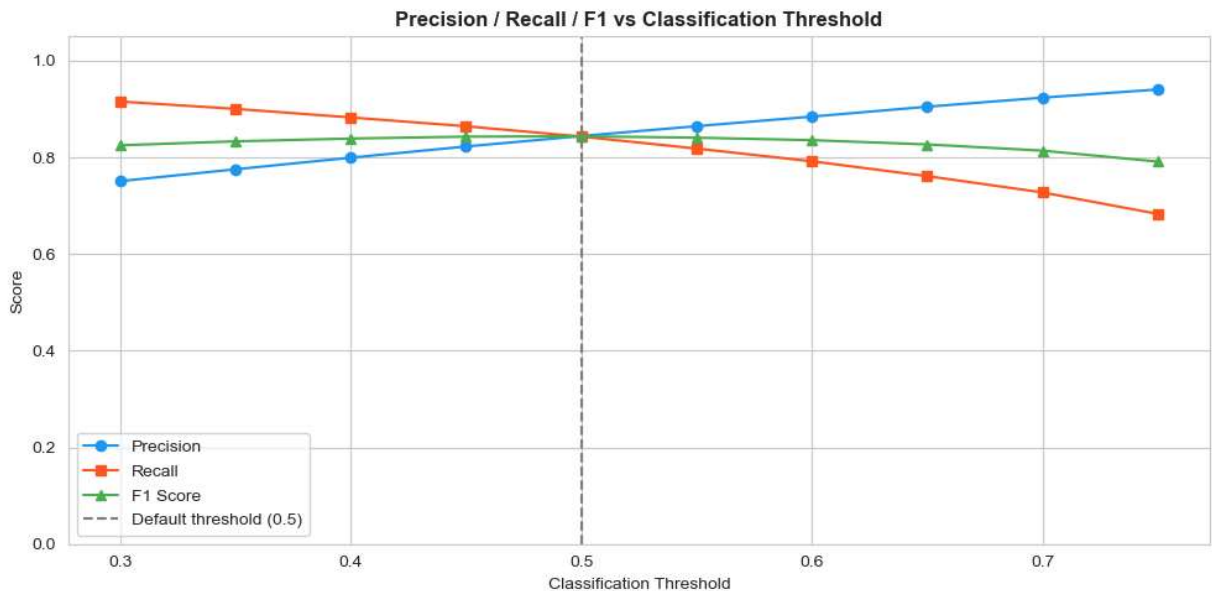
```

for thresh in thresholds_range:
    y_pred_thresh = (y_pred_proba >= thresh).astype(int)
    precisions.append(precision_score(y_test, y_pred_thresh, zero_division=0))
    recalls.append(recall_score(y_test, y_pred_thresh, zero_division=0))
    f1s.append(f1_score(y_test, y_pred_thresh, zero_division=0))

fig, ax = plt.subplots(figsize=(10, 5))
ax.plot(thresholds_range, precisions, 'o-', color='#2196F3', label='Precision')
ax.plot(thresholds_range, recalls, 's-', color='#FF5722', label='Recall')
ax.plot(thresholds_range, f1s, '^-', color='#4CAF50', label='F1 Score')
ax.axvline(0.5, color='gray', linestyle='--', linewidth=1.5, label='Default threshold')
ax.set_xlabel('Classification Threshold')
ax.set_ylabel('Score')
ax.set_title('Precision / Recall / F1 vs Classification Threshold', fontweight='bold')
ax.legend()
ax.set_ylim(0, 1.05)
plt.tight_layout()
plt.show()

print('\nThreshold Sensitivity Table:')
thresh_df = pd.DataFrame({
    'Threshold': thresholds_range,
    'Precision': precisions,
    'Recall': recalls,
    'F1': f1s
})
print(thresh_df.to_string(index=False))

```



Threshold Sensitivity Table:

Threshold	Precision	Recall	F1
0.3000	0.7507	0.9155	0.8250
0.3500	0.7753	0.9003	0.8331
0.4000	0.7993	0.8826	0.8389
0.4500	0.8224	0.8643	0.8429
0.5000	0.8441	0.8430	0.8436
0.5500	0.8645	0.8182	0.8407
0.6000	0.8844	0.7918	0.8355
0.6500	0.9048	0.7611	0.8268
0.7000	0.9236	0.7274	0.8139
0.7500	0.9404	0.6829	0.7912

```
In [27]: # Final summary
print('=' * 55)
print('  LOGISTIC REGRESSION MODEL – FINAL SUMMARY')
print('=' * 55)
print(f' Training samples : {X_train.shape[0]:>10,}')
print(f' Test samples      : {X_test.shape[0]:>10,}')
print(f' Features used       : {X_train.shape[1]:>10}')
print('-' * 55)
print(f' Accuracy           : {accuracy:>10.4f}')
print(f' Precision           : {precision:>10.4f}')
print(f' Recall              : {recall:>10.4f}')
print(f' F1 Score            : {f1:>10.4f}')
print(f' ROC-AUC             : {roc_auc:>10.4f}')
print('=' * 55)
print(f' Top satisfaction driver : {coef_df.iloc[0]["Feature"]}')
print(f' Top dissatisfaction risk: {coef_df.iloc[-1]["Feature"]}')
print('=' * 55)
```

```
=====
LOGISTIC REGRESSION MODEL – FINAL SUMMARY
=====
Training samples :    103,904
Test samples     :     25,976
Features used    :         22
-----
Accuracy         :     0.8289
Precision        :     0.8441
Recall           :     0.8430
F1 Score         :     0.8436
ROC-AUC          :     0.9035
=====
Top satisfaction driver : Inflight entertainment
Top dissatisfaction risk: Customer Type_disloyal Customer
=====
```